



INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO®

Bases de Datos I

RED SOCIAL PASCUALINA

Diseño del modelo lógico de base de datos

Modelo relacional normalizado (1FN, 2FN, 3FN) y diccionario de datos · Tarea 2

Informe detallado paso a paso

Grupo 1

JHON BYRON QUIROZ PEREZ

Docente: Oralia Cortes Grajales

Medellín, septiembre de 2026

Contenido

1. Introducción.....	4
2. Referencia al modelo conceptual aprobado.....	5
2.1 Necesidades y supuestos que se mantienen.....	6
2.2 Del mapa de conceptos a las tablas.....	6
3. Modelo lógico planteado - Red social Pascualina.....	7
4. Proceso de normalización.....	8
4.1 Primera forma normal (1FN).....	8
4.2 Segunda forma normal (2FN).....	8
4.3 Tercera forma normal (3FN).....	9
4.4 Verificación de cada tabla.....	10
5. Diccionario de datos.....	12
5.1 Módulo perfil.....	12
Tabla programa_academico.....	12
Tabla estudiante.....	12
Tabla enlace_perfil.....	13
Tabla interes.....	13
Tabla habilidad.....	14
Tabla estudiante_interes.....	14
Tabla estudiante_habilidad.....	14
5.2 Módulo conexiones.....	15
Tabla seguimiento.....	15
Tabla mentoría.....	15
5.3 Módulo compartido.....	15
Tabla materia.....	15
5.4 Módulo publicaciones.....	16
Tabla publicacion.....	16
Tabla publicacion_etiqueta.....	16
Tabla pregunta.....	17
Tabla recurso.....	17
Tabla noticia.....	17
Tabla meme.....	18
Tabla comentario.....	18
Tabla reaccion.....	18
5.5 Módulo grupos.....	19
Tabla grupo.....	19
Tabla grupo_estudio.....	19
Tabla equipo_hackaton.....	19
Tabla club_interes.....	20
Tabla membresia.....	20
5.6 Módulo eventos.....	20
Tabla evento.....	20
Tabla inscripcion.....	21
5. Conclusiones individuales Jhon Byron Quiroz Pérez.....	22
CONCLUSION.....	22
6. Referencias.....	23

1. Introducción

En la Tarea 1 se diseñó y aprobó el **modelo conceptual** (Entidad-Relación extendido) de la Red Social Pascualina, una plataforma para que los estudiantes creen su perfil, se conecten con compañeros y mentores, publiquen actualizaciones, formen grupos y programen eventos. Ese modelo describe *qué* datos necesita la red social, pero todavía no dice *cómo* se organizarán en tablas.

Este informe presenta el segundo paso del diseño: el **modelo lógico relacional**. Para obtenerlo se aplicaron las reglas de transformación del modelo ER al relacional y el proceso de **normalización** hasta la tercera forma normal (1FN, 2FN y 3FN), siguiendo el libro de Elmasri y Navathe y el recurso «De la idea al plano» de la unidad. El resultado se documenta con un **diccionario de datos** que precisa, para cada tabla, sus claves primarias, foráneas y únicas, los tipos de datos genéricos y la justificación de cada decisión.

El modelo final tiene **25 tablas**, **102 columnas** y **34 claves foráneas**, todas en tercera forma normal. Es independiente del gestor de bases de datos: los tipos de datos son genéricos (texto, entero, lógico, fecha) y su traducción a SQL corresponde a la etapa física.

2. Referencia al modelo conceptual aprobado

El punto de partida es el modelo Entidad-Relación extendido entregado en la **Tarea 1** (carpeta [tarea1/informe](#) del repositorio). La figura 1 lo reproduce y la tabla 2 resume sus elementos.

Modelo Entidad-Relación Extendido (MER / EER) — Red Social Pascualina

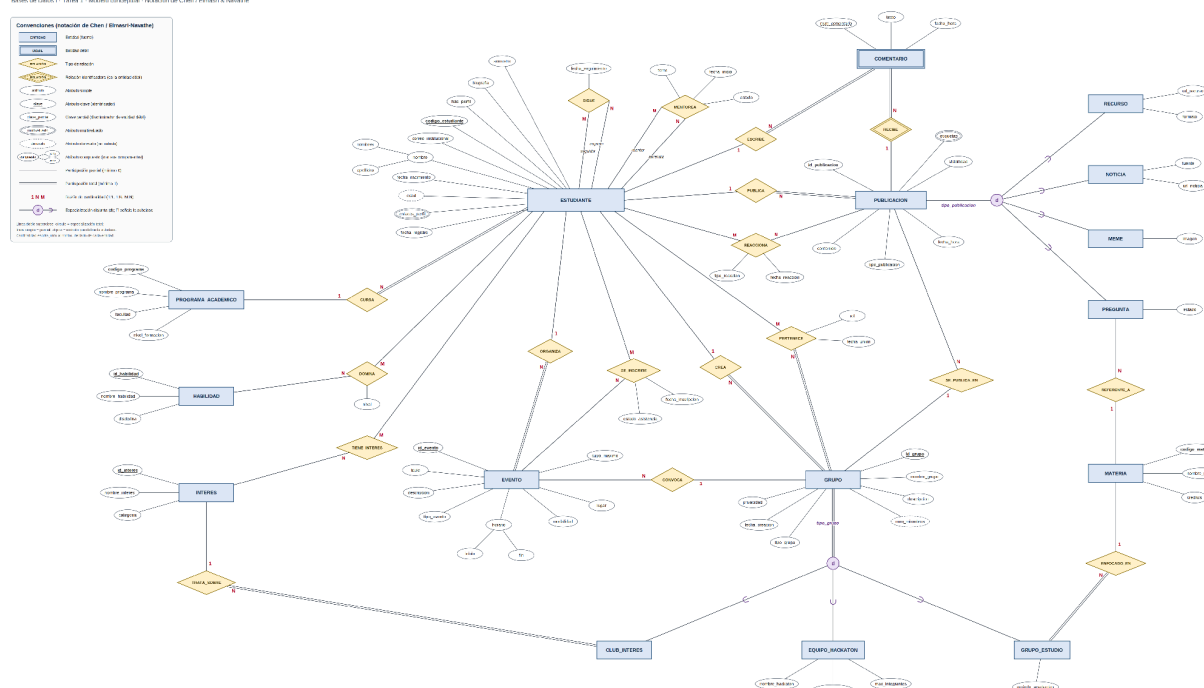


Figura 1. Modelo conceptual (MER / EER) aprobado en la Tarea 1. La versión en alta resolución está en [tarea1/informe/diagrama_MER_RedSocialPascualina.png](#).

Tabla 2. Elementos del modelo conceptual que se transforman.

Elemento conceptual	Cantidad	Detalle
Entidades fuertes	8	ESTUDIANTE, PROGRAMA_ACADEMICO, INTERES, HABILIDAD, MATERIA, PUBLICACION, GRUPO, EVENTO
Subclases	7	PREGUNTA, RECURSO, NOTICIA, MEME (de PUBLICACION); GRUPO_ESTUDIO, EQUIPO_HACKATON, CLUB_INTERES (de GRUPO)
Entidad débil	1	COMENTARIO, identificada por PUBLICACION mediante RECIBE
Relaciones 1:N o N:1	11	CURSA, PUBLICA, ESCRIBE, RECIBE, SE_PUBLICA_EN, REFERENTE_A, CREA, ENFOCADO_EN, TRATA SOBRE, ORGANIZA, CONVOCA
Relaciones M:N	7	TIENE_INTERES, DOMINA, SIGUE, MENTOREA, REACCIONA, PERTENECE, SE_INSCRIBE
Relaciones 1:1	0	No hay relaciones 1:1 entre entidades; solo aparecen en el modelo lógico entre superclase y subclase.
Atributos especiales	6	Compuestos: <i>nombre</i> , <i>horario</i> . Multivaluados: <i>enlaces_perfil</i> , <i>etiquetas</i> . Derivados: <i>edad</i> , <i>num_miembros</i> .

2.1 Necesidades y supuestos que se mantienen

Se revisaron de nuevo las cinco necesidades del enunciado (crear perfiles, conectar con compañeros, publicar actualizaciones, crear y unirse a grupos, y programar eventos). Todas quedaron cubiertas en el modelo conceptual, por lo que el modelo lógico no agrega ni elimina requisitos: solo cambia su forma de representación. También se conservan los supuestos S1 a S12 de la Tarea 1; los que más influyen en el diseño lógico son:

- **S1.** El estudiante se identifica con su código institucional y su correo es único → clave primaria natural y restricción UNIQUE.
- **S5 y S9.** Una publicación y un grupo son de un solo tipo → la columna `tipo_publicacion` o `tipo_grupo` y las tablas de subclase.
- **S6 y S11.** Las publicaciones pueden no estar en un grupo y los eventos pueden ser independientes → claves foráneas que admiten nulos.
- **S7.** Un comentario desaparece con su publicación → eliminación en cascada.
- **S8.** Una reacción por estudiante y publicación → clave primaria compuesta en la tabla `reaccion`.

2.2 Del mapa de conceptos a las tablas

Al revisar el mapa de conceptos con la mirada del modelo lógico, el equipo identificó tres grupos de elementos: **conceptos que serán tablas** (las entidades, las subclases y las relaciones M:N), **conceptos que serán columnas** (los atributos simples y las claves foráneas que implementan las relaciones 1:N) y **conceptos que no se almacenan** (los atributos derivados). Esta clasificación, detallada en las secciones 5 y 6, es la lluvia de ideas de esta tarea.

3. Modelo lógico planteado - Red social Pascualina

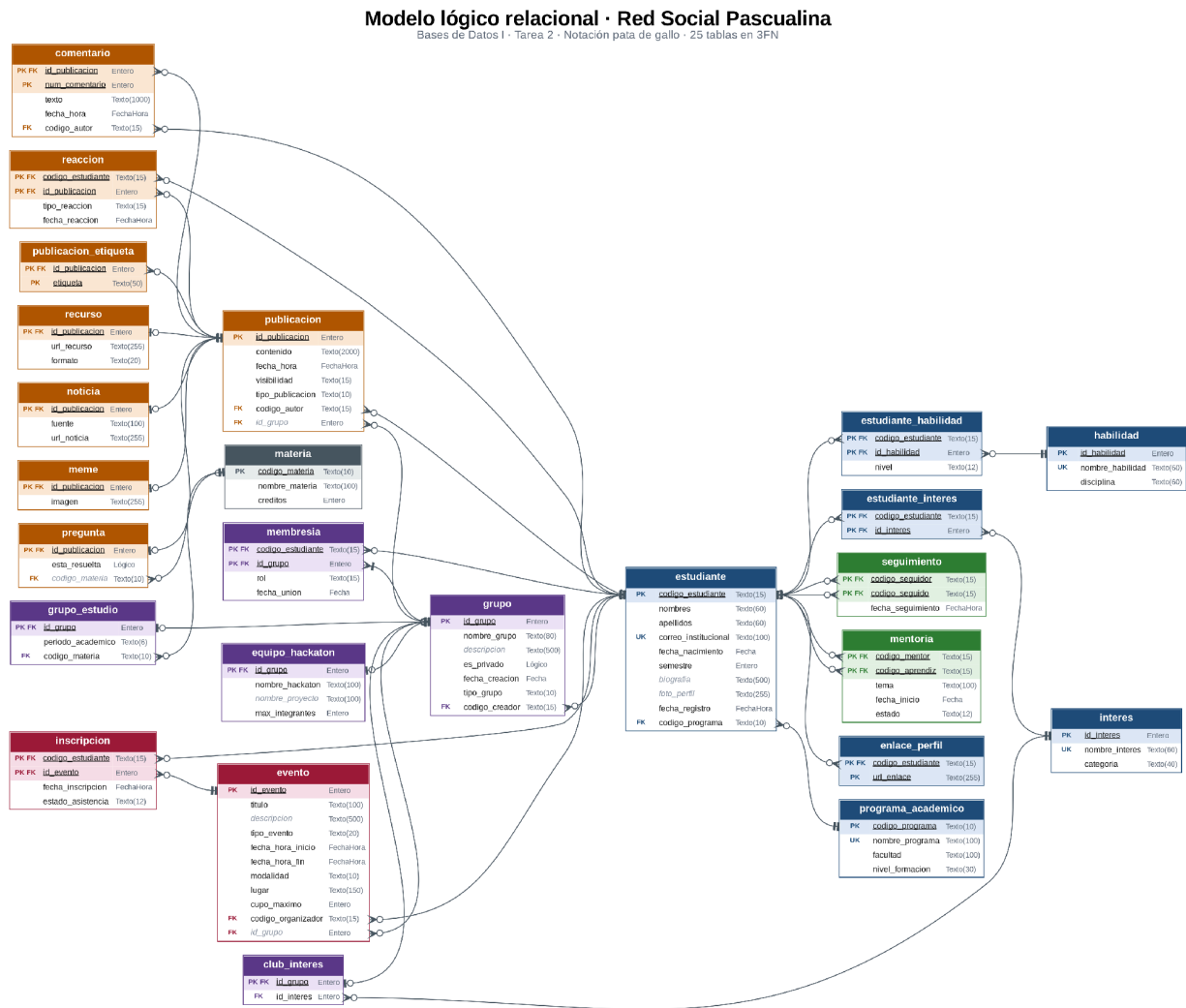


Figura 2. Diagrama del modelo lógico relacional de la Red Social Pascualina (25 tablas en 3FN). La imagen en alta resolución y el archivo editable están en *tarea2/informe*.

4. Proceso de normalización

La **normalización** analiza las **dependencias funcionales** de cada tabla para eliminar la redundancia y las anomalías de inserción, actualización y borrado. Una dependencia funcional $X \rightarrow Y$ significa que el valor de X determina un único valor de Y ; por ejemplo, `codigo_estudiante` $\rightarrow$ `correo_institucional`. A continuación se explica cómo se aplicó cada forma normal, con ejemplos de lo que habría ocurrido si no se hubiera normalizado.

4.1 Primera forma normal (1FN)

Regla: todos los atributos deben tener valores atómicos (indivisibles y de un solo valor) y la tabla debe tener clave primaria. No se admiten atributos compuestos, multivaluados ni grupos repetitivos.

Problema detectado. Si la tabla de estudiantes se hubiera construido directamente desde el diagrama, tendría un nombre compuesto y una lista de enlaces en una misma celda:

Tabla 6. Ejemplo que NO cumple la 1FN.

codigo_estudiante	nombre	enlaces_perfil
20261001	Ana María Gómez Ruiz	github.com/anag, linkedin.com/in/anag
20261002	Luis Pérez Toro	github.com/lperez

Solución. El nombre se separa en `nombres` y `apellidos`, y los enlaces pasan a la tabla `enlace_perfil`, con una fila por enlace:

Tabla 7. Resultado en 1FN: `estudiante` (extracto) y `enlace_perfil`.

codigo_estudiante	nombres	apellidos	codigo_estudiante	url_enlace
20261001	Ana María	Gómez Ruiz	20261001	github.com/anag
20261002	Luis	Pérez Toro	20261001	linkedin.com/in/anag
			20261002	github.com/lperez

Lo mismo se hizo con `etiquetas` (tabla `publicacion_etiqueta`) y con el atributo compuesto `horario` del evento (`fecha_hora_inicio` y `fecha_hora_fin`). Además, cada tabla recibió una clave primaria. Con estos cambios **las 25 tablas cumplen la 1FN**.

4.2 Segunda forma normal (2FN)

Regla: la tabla debe estar en 1FN y cada atributo que no forma parte de la clave debe depender de **toda** la clave primaria, no solo de una parte. Solo puede incumplirse en tablas con clave compuesta; las que tienen clave simple están automáticamente en 2FN.

Problema detectado. En una primera versión de las inscripciones se pensó mostrar el título y el cupo del evento junto a cada inscrito:

Tabla 8. Ejemplo que NO cumple la 2FN: `título` y `cupos_maximo` dependen solo de `id_evento`.

codigo_estudiante (PK)	id_evento (PK)	fecha_inscripcion	título	cupos_maximo
20261001	7	2026-09-20 10:15	Taller de SQL	30
20261002	7	2026-09-20 11:02	Taller de SQL	30
20261002	9	2026-09-21 08:40	Cine foro	50

`titulo` y `cupo_maximo` dependen solo de `id_evento` (dependencia parcial), así que se repiten en cada inscripción. Si el título cambiara, habría que actualizar muchas filas y podrían quedar valores distintos para el mismo evento (anomalía de actualización).

Solución. Los datos del evento quedan solo en `evento`, y `inscripcion` conserva únicamente los atributos que dependen de la pareja estudiante–evento: `fecha_inscripcion` y `estado_asistencia`. El mismo análisis se aplicó a las otras tablas con clave compuesta:

- `estudiante_habilidad`: `nivel` depende del estudiante y de la habilidad, así que se queda; `nombre_habilidad` depende solo de `id_habilidad` y se deja en `habilidad`.
- `membresia`: `rol` y `fecha_union` dependen de la pareja estudiante–grupo.
- `comentario`: `texto`, `fecha_hora` y `codigo_autor` dependen de la clave completa (`id_publicacion`, `num_comentario`), porque `num_comentario` solo identifica al comentario dentro de su publicación.
- `seguimiento`, `mentoria` y `reaccion`: sus atributos describen la pareja, no a uno solo de sus miembros.
- `estudiante_interes`, `enlace_perfil` y `publicacion_etiqueta` no tienen atributos fuera de la clave, por lo que cumplen la 2FN de forma trivial.

4.3 Tercera forma normal (3FN)

Regla: la tabla debe estar en 2FN y ningún atributo no clave puede depender de otro atributo no clave (**dependencia transitiva**). Todo atributo debe depender de la clave, de toda la clave y nada más que de la clave.

Problema detectado. Si la tabla de estudiantes guardara también el nombre del programa y la facultad:

Tabla 9. Ejemplo que NO cumple la 3FN: `codigo_estudiante` → `codigo_programa` → `nombre_programa`, `facultad`.

<code>codigo_estudiante</code> (PK)	<code>nombres</code>	<code>codigo_programa</code>	<code>nombre_programa</code>	<code>facultad</code>
20261001	Ana María	TDS	Tecnología en Desarrollo de Software	Ingeniería
20261002	Luis	TDS	Tecnología en Desarrollo de Software	Ingeniería
20261003	Sara	IE	Ingeniería Eléctrica	Ingeniería

`nombre_programa` y `facultad` no dependen del estudiante sino del programa. Se repetirían para cada estudiante y, si no hubiera estudiantes en un programa, no se podría registrar el programa (anomalía de inserción).

Solución. Estos datos se trasladan a `programa_academico`, y `estudiante` conserva solo la clave foránea `codigo_programa`. La misma revisión se hizo en el resto de tablas:

- `grupo_estudio` y `pregunta` guardan `codigo_materia`, pero no `nombre_materia` ni `creditos`, que están en `materia`.
- `club_interes` guarda `id_interes`, pero no `nombre_interes`.
- `evento`, `publicacion`, `grupo` y `comentario` guardan el código del estudiante (organizador, autor, creador), pero nunca su nombre o su correo.
- En `programa_academico`, `facultad` depende directamente del programa. Como la facultad no tiene otros datos, no genera dependencias transitivas y no se separó en otra tabla.

- `correo_institucional` determina a los demás atributos de `estudiante`, pero es una clave candidata (UNIQUE), no un atributo no clave, así que no viola la 3FN.

4.4 Verificación de cada tabla

La tabla 10 resume, para cada tabla, su clave primaria, su dependencia funcional principal y la verificación de las formas normales. Todas cumplen la 1FN porque sus atributos son atómicos y tienen clave primaria.

Tabla 10. Dependencias funcionales y verificación de 2FN y 3FN por tabla.

Tabla	Clave primaria	Dependencia funcional principal	2FN	3FN
<code>programa_academico</code>	<code>codigo_programa</code>	{ <code>codigo_programa</code> } → <code>nombre_programa</code> , <code>facultad</code> , <code>nivel_formacion</code> <code>nombre_programa</code> → <code>codigo_programa</code> (clave candidata)	Sí (clave simple)	Sí
<code>estudiante</code>	<code>codigo_estudiante</code>	{ <code>codigo_estudiante</code> } → <code>nombres</code> , <code>apellidos</code> , <code>correo_institucional</code> , <code>fecha_nacimiento</code> , <code>semestre</code> , <code>biografia</code> , <code>foto_perfil</code> , <code>fecha_registro</code> , <code>codigo_programa</code> <code>correo_institucional</code> → <code>codigo_estudiante</code> (clave candidata)	Sí (clave simple)	Sí
<code>enlace_perfil</code>	<code>codigo_estudiante</code> , <code>url_enlace</code>	Sin atributos fuera de la clave (tabla «toda clave»)	Sí (trivial)	Sí
<code>interes</code>	<code>id_interes</code>	{ <code>id_interes</code> } → <code>nombre_interes</code> , <code>categoria</code> <code>nombre_interes</code> → <code>id_interes</code> (clave candidata)	Sí (clave simple)	Sí
<code>habilidad</code>	<code>id_habilidad</code>	{ <code>id_habilidad</code> } → <code>nombre_habilidad</code> , <code>disciplina</code> <code>nombre_habilidad</code> → <code>id_habilidad</code> (clave candidata)	Sí (clave simple)	Sí
<code>estudiante_interes</code>	<code>codigo_estudiante</code> , <code>id_interes</code>	Sin atributos fuera de la clave (tabla «toda clave»)	Sí (trivial)	Sí
<code>estudiante_habilidad</code>	<code>codigo_estudiante</code> , <code>id_habilidad</code>	{ <code>codigo_estudiante</code> , <code>id_habilidad</code> } → <code>nivel</code>	Sí (dependen de toda la clave)	Sí
<code>seguimiento</code>	<code>codigo_seguidor</code> , <code>codigo_seguido</code>	{ <code>codigo_seguidor</code> , <code>codigo_seguido</code> } → <code>fecha_seguimiento</code>	Sí (dependen de toda la clave)	Sí
<code>mentoria</code>	<code>codigo_mentor</code> , <code>codigo_aprendiz</code>	{ <code>codigo_mentor</code> , <code>codigo_aprendiz</code> } → <code>tema</code> , <code>fecha_inicio</code> , <code>estado</code>	Sí (dependen de toda la clave)	Sí
<code>materia</code>	<code>codigo_materia</code>	{ <code>codigo_materia</code> } → <code>nombre_materia</code> , <code>creditos</code>	Sí (clave simple)	Sí
<code>publicacion</code>	<code>id_publicacion</code>	{ <code>id_publicacion</code> } → <code>contenido</code> , <code>fecha_hora</code> , <code>visibilidad</code> , <code>tipo_publicacion</code> , <code>codigo_autor</code> , <code>id_grupo</code>	Sí (clave simple)	Sí
<code>publicacion_etiqueta</code>	<code>id_publicacion</code> , <code>etiqueta</code>	Sin atributos fuera de la clave (tabla «toda clave»)	Sí (trivial)	Sí
<code>pregunta</code>	<code>id_publicacion</code>	{ <code>id_publicacion</code> } → <code>esta_resuelta</code> , <code>codigo_materia</code>	Sí (clave simple)	Sí
<code>recurso</code>	<code>id_publicacion</code>	{ <code>id_publicacion</code> } → <code>url_recurso</code> , <code>formato</code>	Sí (clave simple)	Sí

Tabla	Clave primaria	Dependencia funcional principal	2FN	3FN
noticia	id_publicacion	{id_publicacion} → fuente, url_noticia	Sí (clave simple)	Sí
meme	id_publicacion	{id_publicacion} → imagen	Sí (clave simple)	Sí
comentario	id_publicacion, num_comentario	{id_publicacion, num_comentario} → texto, fecha_hora, codigo_autor	Sí (dependen de toda la clave)	Sí
reaccion	codigo_estudiante, id_publicacion	{codigo_estudiante, id_publicacion} → tipo_reaccion, fecha_reaccion	Sí (dependen de toda la clave)	Sí
grupo	id_grupo	{id_grupo} → nombre_grupo, descripcion, es_privado, fecha_creacion, tipo_grupo, codigo_creador	Sí (clave simple)	Sí
grupo_estudio	id_grupo	{id_grupo} → periodo_academico, codigo_materia	Sí (clave simple)	Sí
equipo_hackaton	id_grupo	{id_grupo} → nombre_hackaton, nombre_proyecto, max_integrantes	Sí (clave simple)	Sí
club_interes	id_grupo	{id_grupo} → id_interes	Sí (clave simple)	Sí
membresia	codigo_estudiante, id_grupo	{codigo_estudiante, id_grupo} → rol, fecha_union	Sí (dependen de toda la clave)	Sí
evento	id_evento	{id_evento} → titulo, descripcion, tipo_evento, fecha_hora_inicio, fecha_hora_fin, modalidad, lugar, cupo_maximo, codigo_organizador, id_grupo	Sí (clave simple)	Sí
inscripcion	codigo_estudiante, id_evento	{codigo_estudiante, id_evento} → fecha_inscripcion, estado_asistencia	Sí (dependen de toda la clave)	Sí

5. Diccionario de datos

El diccionario describe cada tabla del modelo lógico. Para cada una se indican su origen en el modelo conceptual, sus claves y una tabla descriptiva con el **nombre del atributo**, el **tipo de dato** y la **longitud**, las **restricciones**, la **descripción** y la **justificación de la elección**. Las tablas se presentan agrupadas por módulo, en el mismo orden del diagrama.

5.1 Módulo perfil

Tabla `programa_academico`

Programas académicos de la institución (área de estudio del estudiante). **Origen:** Entidad PROGRAMA_ACADEMICO.

Clave primaria: `codigo_programa`. **Claves foráneas:** ninguna. **Claves únicas:** `nombre_programa`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_programa</code>	Texto(10)	PK	Código institucional del programa.	Clave natural heredada del modelo conceptual. Es texto porque puede incluir letras o ceros a la izquierda y nunca se usa en cálculos.
<code>nombre_programa</code>	Texto(100)	UK NOT NULL	Nombre oficial del programa.	Único para evitar programas duplicados; 100 caracteres cubren los nombres más largos.
<code>facultad</code>	Texto(100)	NOT NULL	Facultad a la que pertenece el programa.	Solo sirve para agrupar y no tiene datos propios, por lo que no se separa en otra tabla (sección 6.4).
<code>nivel_formacion</code>	Texto(30)	NOT NULL CHECK (valores permitidos)	Técnico profesional, Tecnológico o Profesional universitario.	Dominio cerrado de tres valores, controlado con una restricción CHECK.

Tabla `estudiante`

Estudiantes registrados en la red social con los datos de su perfil. **Origen:** Entidad ESTUDIANTE + relación CURSA.

Clave primaria: `codigo_estudiante`. **Claves foráneas:** `codigo_programa` → `programa_academico.codigo_programa`. **Claves únicas:** `correo_institucional`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_estudiante</code>	Texto(15)	PK	Código estudiantil asignado por la institución.	Clave natural del modelo conceptual: única y estable. Texto porque es un identificador, no una cantidad.
<code>nombres</code>	Texto(60)	NOT NULL	Nombres del estudiante.	Componente del atributo compuesto nombre; se separa para cumplir la 1FN y poder ordenar por apellidos.
<code>apellidos</code>	Texto(60)	NOT NULL	Apellidos del estudiante.	Segundo componente del atributo compuesto nombre (1FN).
<code>correo_institucional</code>	Texto(100)	UK NOT NULL	Correo electrónico institucional.	Clave candidata: la restricción UNIQUE impide que dos estudiantes compartan correo.
<code>fecha_nacimiento</code>	Fecha	NOT NULL	Fecha de nacimiento.	Dato estable del que se calcula la edad, atributo derivado que no se almacena.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>semestre</code>	Entero	NOT NULL CHECK BETWEEN 1 AND 10	Semestre que cursa (1 a 10).	Entero porque se compara: un mentor debe estar en un semestre superior al del aprendiz.
<code>biografia</code>	Texto(500)	NULL	Texto breve de presentación del perfil.	Opcional; 500 caracteres bastan para una presentación corta.
<code>foto_perfil</code>	Texto(255)	NULL	Ruta o URL de la foto de perfil.	Se guarda la ruta del archivo y no la imagen; 255 es la longitud habitual de una URL.
<code>fecha_registro</code>	FechaHora	NOT NULL DEFAULT fecha y hora actual	Momento en que se creó el perfil.	Se llena por defecto con la fecha y hora actuales.
<code>codigo_programa</code>	Texto(10)	FK → programa_academico NOT NULL	Programa académico que cursa.	Implementa CURSA (N:1). Es obligatoria (NOT NULL) porque la participación del estudiante es total.

Tabla `enlace_perfil`

Enlaces externos del perfil (GitHub, LinkedIn, portafolio). **Origen:** Atributo multivaluado `enlaces_perfil`.

Clave primaria: `codigo_estudiante`, `url_enlace` (compuesta). **Claves foráneas:** `codigo_estudiante` → `estudiante.codigo_estudiante`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_estudiante</code>	Texto(15)	PK FK → estudiante	Estudiante dueño del enlace.	Parte de la clave primaria y clave foránea hacia estudiante, según la regla del atributo multivaluado.
<code>url_enlace</code>	Texto(255)	PK	Dirección del enlace.	Parte de la clave: un estudiante no puede registrar dos veces el mismo enlace.

Tabla `interes`

Catálogo de intereses (inteligencia artificial, videojuegos, fútbol...). **Origen:** Entidad INTERES.

Clave primaria: `id_interes`. **Claves foráneas:** ninguna. **Claves únicas:** `nombre_interes`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_interes</code>	Entero	PK	Identificador del interés.	Clave sustituta autoincremental: el interés no tiene un código natural.
<code>nombre_interes</code>	Texto(60)	UK NOT NULL	Nombre del interés.	Único para mantener el catálogo sin duplicados.
<code>categoria</code>	Texto(40)	NOT NULL	Categoría del interés: tecnología, deporte, arte, ocio...	Agrupar los intereses para hacer recomendaciones.

Tabla habilidad

Catálogo de habilidades en disciplinas o deportes. **Origen:** Entidad HABILIDAD.

Clave primaria: `id_habilidad`. **Claves foráneas:** ninguna. **Claves únicas:** `nombre_habilidad`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_habilidad</code>	Entero	PK	Identificador de la habilidad.	Clave sustituta autoincremental.
<code>nombre_habilidad</code>	Texto(60)	UK NOT NULL	Nombre de la habilidad (Python, diseño gráfico, voleibol...).	Única para que la misma habilidad no se escriba de formas distintas.
<code>disciplina</code>	Texto(60)	NOT NULL	Disciplina o deporte al que pertenece la habilidad.	Responde a la necesidad de mostrar habilidades en disciplinas o deportes.

Tabla estudiante_interes

Intereses registrados por cada estudiante. **Origen:** Relación TIENE_INTERES (M:N).

Clave primaria: `codigo_estudiante, id_interes` (compuesta). **Claves foráneas:** `codigo_estudiante` → `estudiante.codigo_estudiante`; `id_interes` → `interes.id_interes`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_estudiante</code>	Texto(15)	PK FK → estudiante	Estudiante.	Parte de la clave compuesta; referencia a estudiante.
<code>id_interes</code>	Entero	PK FK → interes	Interés registrado.	Parte de la clave compuesta; referencia a interes.

Tabla estudiante_habilidad

Habilidades que domina cada estudiante y su nivel. **Origen:** Relación DOMINA (M:N).

Clave primaria: `codigo_estudiante, id_habilidad` (compuesta). **Claves foráneas:** `codigo_estudiante` → `estudiante.codigo_estudiante`; `id_habilidad` → `habilidad.id_habilidad`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_estudiante</code>	Texto(15)	PK FK → estudiante	Estudiante.	Parte de la clave compuesta; referencia a estudiante.
<code>id_habilidad</code>	Entero	PK FK → habilidad	Habilidad.	Parte de la clave compuesta; referencia a habilidad.
<code>nivel</code>	Texto(12)	NOT NULL CHECK (valores permitidos)	Básico, Intermedio o Avanzado.	Atributo de la relación: depende de las dos columnas de la clave a la vez (cumple 2FN).

5.2 Módulo conexiones

Tabla seguimiento

Quién sigue a quién dentro de la red. **Origen:** Relación recursiva SIGUE (M:N).

Clave primaria: `codigo_seguidor`, `codigo_seguido` (compuesta). **Claves foráneas:** `codigo_seguidor` → `estudiante.codigo_estudiante`; `codigo_seguido` → `estudiante.codigo_estudiante`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_seguidor</code>	Texto(15)	PK FK → estudiante	Estudiante que sigue (rol seguidor).	El nombre del rol distingue las dos claves foráneas que apuntan a estudiante.
<code>codigo_seguido</code>	Texto(15)	PK FK → estudiante	Estudiante seguido (rol seguido).	Segunda clave foránea hacia estudiante.
<code>fecha_seguimiento</code>	FechaHora	NOT NULL	Momento en que empezó a seguirlo.	Atributo de la relación: describe a la pareja.

Restricción de tabla: `codigo_seguidor` <> `codigo_seguido` (nadie se sigue a sí mismo).

Tabla mentoría

Mentorías entre estudiantes de semestres avanzados y sus aprendices. **Origen:** Relación recursiva MENTOREA (M:N).

Clave primaria: `codigo_mentor`, `codigo_aprendiz` (compuesta). **Claves foráneas:** `codigo_mentor` → `estudiante.codigo_estudiante`; `codigo_aprendiz` → `estudiante.codigo_estudiante`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_mentor</code>	Texto(15)	PK FK → estudiante	Estudiante mentor.	Rol mentor; clave foránea hacia estudiante.
<code>codigo_aprendiz</code>	Texto(15)	PK FK → estudiante	Estudiante aprendiz.	Rol aprendiz; clave foránea hacia estudiante.
<code>tema</code>	Texto(100)	NOT NULL	Tema o materia de la mentoría.	Atributo de la relación.
<code>fecha_inicio</code>	Fecha	NOT NULL	Fecha en que comenzó la mentoría.	Atributo de la relación.
<code>estado</code>	Texto(12)	NOT NULL CHECK (valores permitidos)	Activa o Finalizada.	Dominio cerrado controlado con CHECK.

Restricción de tabla: `codigo_mentor` <> `codigo_aprendiz`.

5.3 Módulo compartido

Tabla materia

Asignaturas del plan de estudios. **Origen:** Entidad MATERIA.

Clave primaria: `codigo_materia`. **Claves foráneas:** ninguna.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_materia</code>	Texto(10)	PK	Código oficial de la asignatura.	Clave natural; texto porque es un identificador.
<code>nombre_materia</code>	Texto(100)	NOT NULL	Nombre de la asignatura.	100 caracteres cubren los nombres más largos.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
creditos	Entero	NOT NULL CHECK > 0	Créditos académicos.	Entero porque los créditos no tienen decimales.

5.4 Módulo publicaciones

Tabla `publicacion`

Publicaciones de los estudiantes (datos comunes a todos los tipos). **Origen:** Superclase PUBLICACION + relaciones PUBLICA y SE_PUBLICA_EN.

Clave primaria: `id_publicacion`. **Claves foráneas:** `codigo_autor` → `estudiante.codigo_estudiante`; `id_grupo` → `grupo.id_grupo`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_publicacion</code>	Entero	PK	Identificador de la publicación.	Clave sustituta autoincremental.
<code>contenido</code>	Texto(2000)	NOT NULL	Texto de la publicación.	2000 caracteres permiten publicaciones extensas sin llegar a ser un documento.
<code>fecha_hora</code>	FechaHora	NOT NULL	Momento de la publicación.	Permite ordenar el muro cronológicamente.
<code>visibilidad</code>	Texto(15)	NOT NULL CHECK (valores permitidos)	Pública, Seguidores o Grupo.	Dominio cerrado controlado con CHECK.
<code>tipo_publicacion</code>	Texto(10)	NOT NULL CHECK (valores permitidos)	General, Pregunta, Recurso, Noticia o Meme.	Atributo de definición de la especialización: indica en qué tabla de subclase están los datos propios.
<code>codigo_autor</code>	Texto(15)	FK → estudiante NOT NULL	Estudiante autor.	Implementa PUBLICA (1:N); obligatoria porque toda publicación tiene autor.
<code>id_grupo</code>	Entero	FK → grupo NULL	Grupo en el que se publicó.	Implementa SE_PUBLICA_EN (N:1). Admite nulos: la publicación puede ir al muro general (participación parcial).

Tabla `publicacion_etiqueta`

Etiquetas (#BD1, #Python...) de cada publicación. **Origen:** Atributo multivaluado etiquetas.

Clave primaria: `id_publicacion`, `etiqueta` (compuesta). **Claves foráneas:** `id_publicacion` → `publicacion.id_publicacion`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_publicacion</code>	Entero	PK FK → publicacion	Publicación etiquetada.	Parte de la clave; referencia a publicacion.
<code>etiqueta</code>	Texto(50)	PK	Texto de la etiqueta, sin el símbolo #.	Parte de la clave: una etiqueta no se repite en la misma publicación.

Tabla pregunta

Preguntas sobre tareas. **Origen:** Subclase PREGUNTA + relación REFERENTE_A.

Clave primaria: `id_publicacion`. **Claves foráneas:** `id_publicacion` → `publicacion.id_publicacion`; `codigo_materia` → `materia.codigo_materia`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_publicacion</code>	Entero	PK FK → <code>publicacion</code>	Misma clave de la publicación general.	Es a la vez clave primaria y foránea: cada fila corresponde a exactamente una publicación (relación 1:1).
<code>esta_resuelta</code>	Lógico	NOT NULL DEFAULT falso	Verdadero si la pregunta ya fue respondida.	El atributo estado solo tenía dos valores (abierta o resuelta), por eso se representa con el tipo lógico.
<code>codigo_materia</code>	Texto(10)	FK → <code>materia</code> NULL	Materia sobre la que trata la pregunta.	Implementa REFERENTE_A (N:1). Admite nulos porque hay preguntas generales.

Tabla recurso

Recursos útiles compartidos. **Origen:** Subclase RECURSO.

Clave primaria: `id_publicacion`. **Claves foráneas:** `id_publicacion` → `publicacion.id_publicacion`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_publicacion</code>	Entero	PK FK → <code>publicacion</code>	Misma clave de la publicación general.	Es a la vez clave primaria y foránea: cada fila corresponde a exactamente una publicación (relación 1:1).
<code>url_recurso</code>	Texto(255)	NOT NULL	Enlace al material.	Longitud habitual de una URL.
<code>formato</code>	Texto(20)	NOT NULL CHECK (valores permitidos)	PDF, Video, Repositorio, Página web u Otro.	Dominio cerrado controlado con CHECK.

Tabla noticia

Noticias de la industria tecnológica. **Origen:** Subclase NOTICIA.

Clave primaria: `id_publicacion`. **Claves foráneas:** `id_publicacion` → `publicacion.id_publicacion`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_publicacion</code>	Entero	PK FK → <code>publicacion</code>	Misma clave de la publicación general.	Es a la vez clave primaria y foránea: cada fila corresponde a exactamente una publicación (relación 1:1).
<code>fuentes</code>	Texto(100)	NOT NULL	Medio que publicó la noticia.	Permite citar el origen de la información.
<code>url_noticia</code>	Texto(255)	NOT NULL	Enlace a la nota original.	Longitud habitual de una URL.

Tabla meme

Memes de la vida universitaria. **Origen:** Subclase MEME.

Clave primaria: `id_publicacion`. **Claves foráneas:** `id_publicacion` → `publicacion.id_publicacion`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_publicacion</code>	Entero	PK FK → <code>publicacion</code>	Misma clave de la publicación general.	Es a la vez clave primaria y foránea: cada fila corresponde a exactamente una publicación (relación 1:1).
<code>imagen</code>	Texto(255)	NOT NULL	Ruta de la imagen del meme.	Se guarda la ruta y no el archivo.

Tabla comentario

Comentarios escritos en las publicaciones. **Origen:** Entidad débil COMENTARIO + relaciones RECIBE y ESCRIBE.

Clave primaria: `id_publicacion`, `num_comentario` (compuesta). **Claves foráneas:** `id_publicacion` → `publicacion.id_publicacion`; `codigo_autor` → `estudiante.codigo_estudiante`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_publicacion</code>	Entero	PK FK → <code>publicacion</code>	Publicación comentada.	Clave de la entidad propietaria: implementa la relación identificadora RECIBE y forma parte de la clave primaria.
<code>num_comentario</code>	Entero	PK	Consecutivo del comentario dentro de la publicación.	Clave parcial de la entidad débil; junto con <code>id_publicacion</code> identifica el comentario.
<code>texto</code>	Texto(1000)	NOT NULL	Contenido del comentario.	Los comentarios son más breves que las publicaciones.
<code>fecha_hora</code>	FechaHora	NOT NULL	Momento en que se escribió.	Permite ordenar los comentarios.
<code>codigo_autor</code>	Texto(15)	FK → <code>estudiante</code> NOT NULL	Estudiante autor del comentario.	Implementa ESCRIBE (1:N); obligatoria.

Tabla reaccion

Reacciones de los estudiantes a las publicaciones. **Origen:** Relación REACCIONA (M:N).

Clave primaria: `codigo_estudiante`, `id_publicacion` (compuesta). **Claves foráneas:**

`codigo_estudiante` → `estudiante.codigo_estudiante`; `id_publicacion` → `publicacion.id_publicacion`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>codigo_estudiante</code>	Texto(15)	PK FK → <code>estudiante</code>	Estudiante que reacciona.	Parte de la clave compuesta, que garantiza una sola reacción por estudiante y publicación (regla R3).
<code>id_publicacion</code>	Entero	PK FK → <code>publicacion</code>	Publicación a la que reacciona.	Parte de la clave compuesta.
<code>tipo_reaccion</code>	Texto(15)	NOT NULL CHECK (valores permitidos)	Me gusta, Útil, Divertido o Me encanta.	Dominio cerrado controlado con CHECK.
<code>fecha_reaccion</code>	FechaHora	NOT NULL	Momento de la reacción.	Atributo de la relación.

5.5 Módulo grupos

Tabla grupo

Grupos creados en la red (datos comunes a los tres tipos). **Origen:** Superclase GRUPO + relación CREA.

Clave primaria: `id_grupo`. **Claves foráneas:** `codigo_creador` → `estudiante.codigo_estudiante`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_grupo</code>	Entero	PK	Identificador del grupo.	Clave sustituta autoincremental.
<code>nombre_grupo</code>	Texto(80)	NOT NULL	Nombre visible del grupo.	Longitud suficiente para nombres descriptivos.
<code>descripcion</code>	Texto(500)	NULL	Propósito del grupo.	Opcional.
<code>es_privado</code>	Lógico	NOT NULL DEFAULT falso	Verdadero si unirse requiere aprobación.	El atributo privacidad tenía dos valores (público o privado), por eso se convierte en lógico.
<code>fecha_creacion</code>	Fecha	NOT NULL	Fecha de creación del grupo.	Se registra una sola vez.
<code>tipo_grupo</code>	Texto(10)	NOT NULL CHECK (valores permitidos)	Estudio, Hackaton o Club.	Atributo de definición de la especialización total: todo grupo debe tener uno de los tres valores.
<code>codigo_creador</code>	Texto(15)	FK → estudiante NOT NULL	Estudiante que creó el grupo.	Implementa CREA (1:N); obligatoria porque todo grupo tiene creador.

Tabla grupo_estudio

Grupos de estudio por materia. **Origen:** Subclase GRUPO_ESTUDIO + relación ENFOCADO_EN.

Clave primaria: `id_grupo`. **Claves foráneas:** `id_grupo` → `grupo.id_grupo`; `codigo_materia` → `materia.codigo_materia`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_grupo</code>	Entero	PK FK → grupo	Misma clave del grupo general.	Clave primaria y foránea (relación 1:1 con grupo).
<code>periodo_academico</code>	Texto(6)	NOT NULL	Periodo en formato AAAA-P (por ejemplo, 2026-2).	Texto porque combina año y periodo con un guion.
<code>codigo_materia</code>	Texto(10)	FK → materia NOT NULL	Materia en la que se enfoca el grupo.	Implementa ENFOCADO_EN (N:1); obligatoria por la participación total.

Tabla equipo_hackaton

Equipos para hackatones. **Origen:** Subclase EQUIPO_HACKATON.

Clave primaria: `id_grupo`. **Claves foráneas:** `id_grupo` → `grupo.id_grupo`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
<code>id_grupo</code>	Entero	PK FK → grupo	Misma clave del grupo general.	Clave primaria y foránea (relación 1:1 con grupo).
<code>nombre_hackaton</code>	Texto(100)	NOT NULL	Hackatón en la que participa.	Dato propio de la subclase.
<code>nombre_proyecto</code>	Texto(100)	NULL	Proyecto que desarrolla el equipo.	Opcional: puede no estar definido al crear el equipo.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
max_integrantes	Entero	NOT NULL CHECK > 0	Número máximo de integrantes.	Entero positivo controlado con CHECK.

Tabla club_interes

Clubes de interés (videojuegos, programación competitiva...). **Origen:** Subclase CLUB_INTERES + relación TRATA SOBRE.

Clave primaria: id_grupo. **Claves foráneas:** id_grupo → grupo.id_grupo; id_interes → interes.id_interes.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
id_grupo	Entero	PK FK → grupo	Misma clave del grupo general.	Clave primaria y foránea (relación 1:1 con grupo).
id_interes	Entero	FK → interes NOT NULL	Interés sobre el que trata el club.	Implementa TRATA SOBRE (N:1); obligatoria por la participación total.

Tabla membresia

Miembros de cada grupo y su rol. **Origen:** Relación PERTENECE (M:N).

Clave primaria: codigo_estudiante, id_grupo (compuesta). **Claves foráneas:** codigo_estudiante → estudiante.codigo_estudiante; id_grupo → grupo.id_grupo.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
codigo_estudiante	Texto(15)	PK FK → estudiante	Estudiante miembro.	Parte de la clave compuesta.
id_grupo	Entero	PK FK → grupo	Grupo.	Parte de la clave compuesta.
rol	Texto(15)	NOT NULL CHECK (valores permitidos)	Administrador, Moderador o Miembro.	Atributo de la relación: la misma persona puede tener roles distintos en grupos distintos.
fecha_union	Fecha	NOT NULL	Fecha en que se unió al grupo.	Atributo de la relación.

5.6 Módulo eventos

Tabla evento

Reuniones de estudio, talleres y actividades sociales. **Origen:** Entidad EVENTO + relaciones ORGANIZA y CONVOCA.

Clave primaria: id_evento. **Claves foráneas:** codigo_organizador → estudiante.codigo_estudiante; id_grupo → grupo.id_grupo.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
id_evento	Entero	PK	Identificador del evento.	Clave sustituta autoincremental.
titulo	Texto(100)	NOT NULL	Nombre del evento.	Longitud suficiente para títulos descriptivos.
descripcion	Texto(500)	NULL	Detalle del evento.	Opcional.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
tipo_evento	Texto(20)	NOT NULL CHECK (valores permitidos)	Reunión de estudio, Taller o Actividad social.	Dominio cerrado controlado con CHECK.
fecha_hora_inicio	FechaHora	NOT NULL	Inicio del evento.	Componente del atributo compuesto horario (1FN).
fecha_hora_fin	FechaHora	NOT NULL	Fin del evento.	Segundo componente de horario; debe ser posterior al inicio (CHECK).
modalidad	Texto(10)	NOT NULL CHECK (valores permitidos)	Presencial, Virtual o Híbrida.	Dominio cerrado controlado con CHECK.
lugar	Texto(150)	NOT NULL	Salón o enlace de la videollamada.	Admite tanto una ubicación física como una URL corta.
cupos_maximo	Entero	NOT NULL CHECK > 0	Número máximo de inscritos.	Entero positivo controlado con CHECK.
codigo_organizador	Texto(15)	FK → estudiante NOT NULL	Estudiante que organiza el evento.	Implementa ORGANIZA (1:N); obligatoria.
id_grupo	Entero	FK → grupo NULL	Grupo que convoca el evento.	Implementa CONVOCA (N:1). Admite nulos porque hay eventos independientes.

Restricción de tabla: `fecha_hora_fin > fecha_hora_inicio`.

Tabla inscripcion

Inscripciones de estudiantes a eventos. **Origen:** Relación SE_INSCRIBE (M:N).

Clave primaria: `codigo_estudiante, id_evento` (compuesta). **Claves foráneas:** `codigo_estudiante → estudiante.codigo_estudiante; id_evento → evento.id_evento`.

Atributo	Tipo y longitud	Restricciones	Descripción	Justificación de la elección
codigo_estudiante	Texto(15)	PK FK → estudiante	Estudiante inscrito.	Parte de la clave compuesta.
id_evento	Entero	PK FK → evento	Evento.	Parte de la clave compuesta.
fecha_inscripcion	FechaHora	NOT NULL	Momento de la inscripción.	Permite atender por orden de llegada si se llena el cupo.
estado_asistencia	Texto(12)	NOT NULL CHECK (valores permitidos) DEFAULT 'Inscrito'	Inscrito, Asistió o No asistió.	Dominio cerrado controlado con CHECK.

5. Conclusiones individuales Jhon Byron Quiroz Pérez

CONCLUSION

Dificultades. Lo más difícil fue convertir las jerarquías de especialización en tablas, porque había varias opciones válidas. Tuve que analizar qué pasaba con los valores nulos y con las claves foráneas en cada caso hasta entender por qué convenía una tabla para la superclase y otra por subclase. También me costó al principio diferenciar una dependencia parcial de una transitiva; los ejemplos con datos (el título del evento repetido en cada inscripción y la facultad repetida en cada estudiante) me ayudaron a verlo con claridad.

Dudas. Me queda la duda de cuándo conviene crear tablas de catálogo para listas como la modalidad o el tipo de evento en lugar de restricciones CHECK, y de cómo se implementarán en SQL las reglas que el modelo lógico no alcanza a garantizar, como el cupo de los eventos.

Aspectos relevantes. Comprendí que un buen modelo conceptual facilita mucho la normalización: como las entidades ya estaban bien separadas, la mayoría de las tablas quedaron en 3FN desde el principio y el proceso sirvió sobre todo para comprobarlo y justificarlo. También aprendí que las decisiones de tipos de datos y nombres no son detalles menores: definir los códigos como texto o nombrar las claves foráneas por su rol hace el modelo más claro y evita errores en la implementación.

6. Referencias

Chen, P. P.-S. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36. <https://doi.org/10.1145/320434.320440>

Elmasri, R. y Navathe, S. B. (2007). *Fundamentos de sistemas de bases de datos* (5.^a ed.). Pearson Addison Wesley. Capítulo 3: El modelo Entidad-Relación; capítulo 4: El modelo Entidad-Relación mejorado.

JGraph Ltd. (s. f.). *draw.io (diagrams.net)* [Software]. <https://www.drawio.com>

Ramez Elmasri . (2007). *Representar para construir* Fundamentos de Sistemas de Bases de Datos . Bases de Datos I, Institución Universitaria Pascual Bravo.